

Strings – Longest Substring without Repeating Characters

Experiment No: 05

Date: 19/09/2026

■ Objective / Aim

To design and implement a Java program that finds the longest substring without repeating characters from a given string, using classes and objects, and applying the sliding window technique.

■ System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

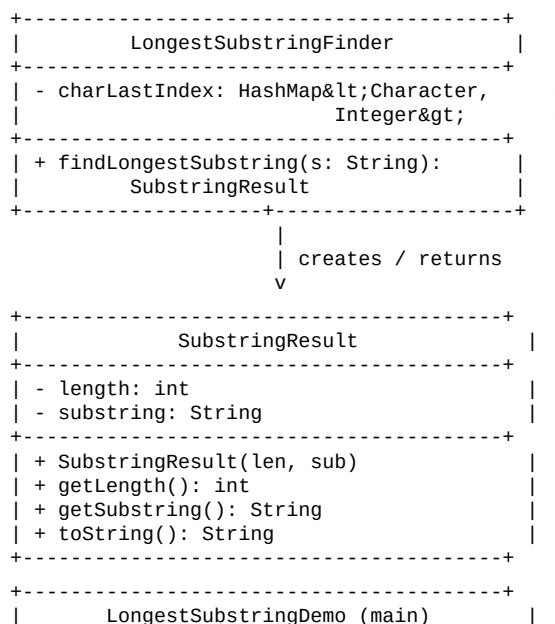
Software

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK 17 or JDK 21)
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

■ Algorithm / Logic Description

1. Create a class **SubstringResult** to hold the outcome: length and substring.
2. Create a class **LongestSubstringFinder** containing the core logic.
3. Define a `HashMap<Character, Integer>` to track the last seen index of each character (sliding window).
4. Initialize two pointers, left and right, both starting at 0, to represent the current window.
5. Traverse the string with right moving from 0 to the end: if the character at right has been seen before and its last index is $\geq$ left, move left to one position after that last seen index (shrink window from the left); update the last seen index of the character at right.
6. Calculate the current window length ($\text{right} - \text{left} + 1$). If it's greater than the maximum found so far, update the maximum length and record the substring.
7. Wrap the final maximum length and substring into a `SubstringResult` object and return it.
8. In `main()`, create a `LongestSubstringFinder` object, test it with multiple input strings, and display each result.

■ UML Class Diagram



```

+-----+
| + main(args: String[]): void |
+-----+
|
| uses (association)
v

```

LongestSubstringFinder ---- creates ----> SubstringResult

Relationship: LongestSubstringDemo uses (association) a LongestSubstringFinder object to process input strings; LongestSubstringFinder creates and returns a SubstringResult object to encapsulate the output (composition – the result object is built fresh each call and has no meaning outside that call).

■ Source Code

```

import java.util.HashMap;

// Result holder class
class SubstringResult {
    private int length;
    private String substring;

    SubstringResult(int length, String substring) {
        this.length = length;
        this.substring = substring;
    }

    public int getLength() {
        return length;
    }

    public String getSubstring() {
        return substring;
    }

    @Override
    public String toString() {
        return "Length = " + length + ", Substring = \"" + substring + "\"";
    }
}

// Core logic class
class LongestSubstringFinder {
    private HashMap<Character, Integer> charLastIndex = new HashMap<>();

    public SubstringResult findLongestSubstring(String s) {
        charLastIndex.clear();

        int left = 0;
        int maxLength = 0;
        int maxStart = 0;

        for (int right = 0; right < s.length(); right++) {
            char currentChar = s.charAt(right);

            if (charLastIndex.containsKey(currentChar)
                && charLastIndex.get(currentChar) >= left) {
                left = charLastIndex.get(currentChar) + 1;
            }

            charLastIndex.put(currentChar, right);

            int currentLength = right - left + 1;

            if (currentLength > maxLength) {
                maxLength = currentLength;
                maxStart = left;
            }
        }

        String longestSubstring = s.substring(maxStart, maxStart + maxLength);
        return new SubstringResult(maxLength, longestSubstring);
    }
}

```

```
// Main class
public class LongestSubstringDemo {
    public static void main(String[] args) {

        LongestSubstringFinder finder = new LongestSubstringFinder();

        String[] testInputs = {
            "abcabcbb",
            "bbbbbb",
            "pwwkew",
            "dvdvf",
            "anviaj"
        };

        for (String input : testInputs) {
            SubstringResult result = finder.findLongestSubstring(input);
            System.out.println("Input : \"" + input + "\"");
            System.out.println("Result : " + result);
            System.out.println("-----");
        }
    }
}
```

■ Compilation & Execution Commands

Save the source code exactly as:

LongestSubstringDemo.java

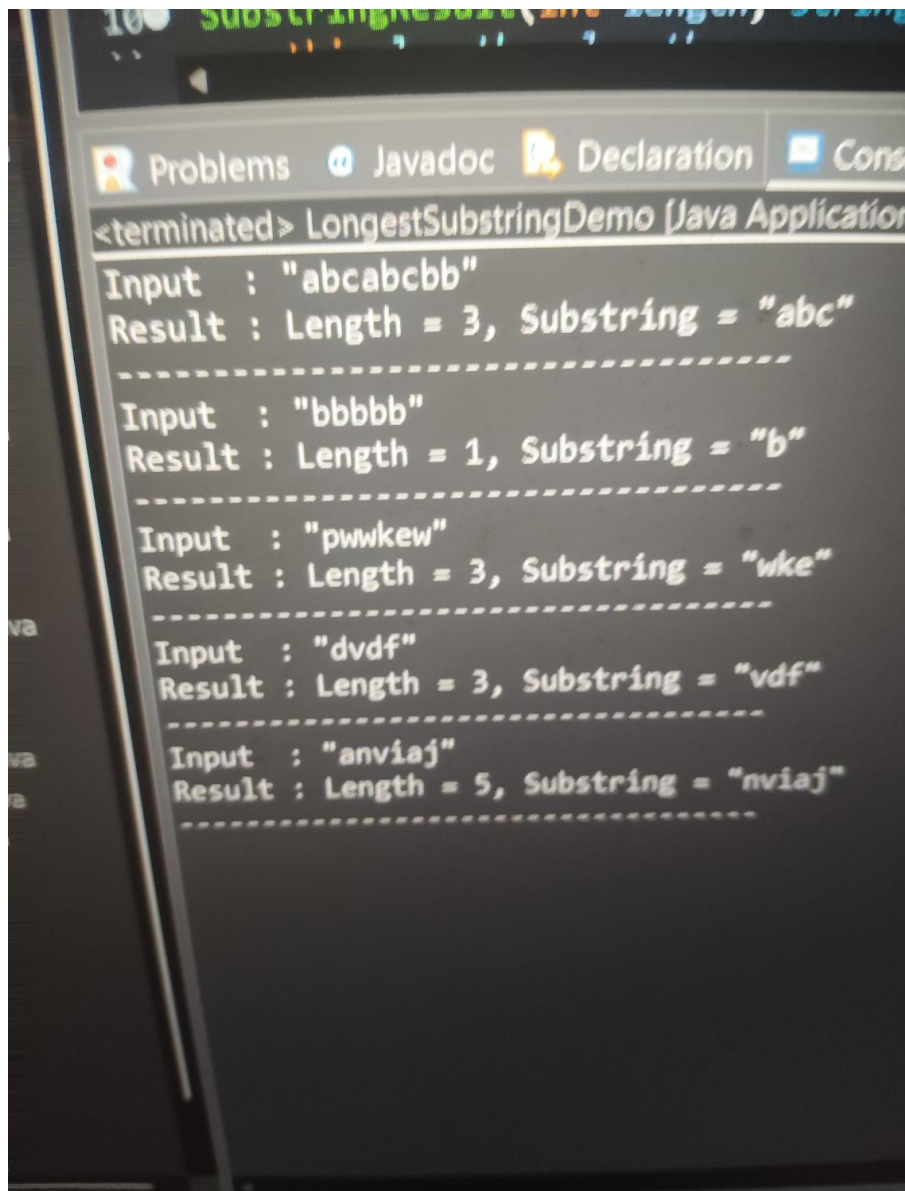
Compile the program:

```
javac LongestSubstringDemo.java
```

Run the compiled program:

```
java LongestSubstringDemo
```

■ Output Screen



```
<terminated> LongestSubstringDemo (Java Application)
Input  : "abcabcbb"
Result : Length = 3, Substring = "abc"
-----
Input  : "bbbbbb"
Result : Length = 1, Substring = "b"
-----
Input  : "pwwkew"
Result : Length = 3, Substring = "wke"
-----
Input  : "dvdvf"
Result : Length = 3, Substring = "vdf"
-----
Input  : "anviaj"
Result : Length = 5, Substring = "nviaj"
-----
```

Figure: Console output of LongestSubstringDemo showing results for all test inputs

■ Result

The Longest Substring without Repeating Characters program was successfully implemented in Java using the sliding window technique. The LongestSubstringFinder class computes the result, and the SubstringResult class encapsulates the output, demonstrating clean separation of logic and data through classes and objects.